

# ResolveAI

## Solution-Oriented Database Schema Design

### Internship Project Technical Design Document

#### Technology Stack

Django Web Portal + AdminLTE2 | FastAPI AI/ML Services | MySQL | Docker | GitLab | Jira

#### Purpose

This document defines the recommended MySQL database architecture for ResolveAI. It is intended to be shared with the internship team as the implementation blueprint. It explains what data must be stored, why each table exists, how entities relate, how the support-ticket lifecycle is represented, and how Django and FastAPI should interact with the database.

Core architecture rule: Django is the system of record for users, RBAC, tickets, messages, assignments, statuses, audit records and business rules. FastAPI AI/ML services receive the minimum required ticket content and return analysis results; they should not directly control ticket status, assignment or closure.

# 1. Database Design Goals

- Single source of truth for all business data.
- Traceability of status changes, assignments, messages and important actions.
- Separation of AI recommendations from final human-controlled business values.
- Historical integrity: events are preserved rather than overwritten.
- Extensibility for departments, SLAs, notifications, attachments and future AI models.
- Security through application-layer RBAC and least-privilege access.
- Efficient reporting using normalized tables and justified indexes.

# 2. High-Level Architecture



Recommended boundary: FastAPI should not directly update core ticket workflow tables in the MVP. Django owns persistence and business rules.

# 3. Database Modules

| Module            | Primary Tables                                                      | Purpose                            |
|-------------------|---------------------------------------------------------------------|------------------------------------|
| Identity and RBAC | users, roles, permissions, user_roles, role_permissions             | Authentication and authorization   |
| Organization      | departments                                                         | Department ownership and queues    |
| Ticket Management | tickets, ticket_messages, ticket_status_history, ticket_assignments | Core support workflow              |
| Classification    | ticket_categories                                                   | Controlled categorization          |
| AI                | ai_analyses, ai_suggestions                                         | AI predictions and recommendations |
| Knowledge Base    | knowledge_articles, knowledge_article_versions                      | FAQ/RAG source content             |
| Attachments       | ticket_attachments                                                  | Uploaded-file metadata             |
| Audit             | audit_logs                                                          | Traceability of sensitive actions  |
| Notifications     | notifications                                                       | In-app notification records        |

# 4. Conceptual Entity Relationship Overview



## 5. Identity and RBAC Schema

Use a custom Django user model from the start. RBAC should be role-based rather than implemented as hard-coded flags scattered through views and templates.

### 5.1 users

| Column     | Type            | Rules / Purpose                         |
|------------|-----------------|-----------------------------------------|
| id         | BIGINT UNSIGNED | Primary key; auto increment             |
| username   | VARCHAR(150)    | Unique login identifier                 |
| email      | VARCHAR(254)    | Unique; normalized to lowercase         |
| password   | VARCHAR(128)    | Django password hash only               |
| first_name | VARCHAR(150)    | Display name                            |
| last_name  | VARCHAR(150)    | Display name                            |
| is_active  | BOOLEAN         | Disable access without deleting history |
| is_staff   | BOOLEAN         | Optional Django admin access flag       |
| last_login | DATETIME NULL   | Last successful login                   |
| created_at | DATETIME        | Creation timestamp                      |
| updated_at | DATETIME        | Last update timestamp                   |

Recommendation: use a custom Django user model from the beginning. Migrating from the default user model later is unnecessarily difficult for an internship project.

### 5.2 roles

| Column                  | Type            | Purpose                      |
|-------------------------|-----------------|------------------------------|
| id                      | BIGINT UNSIGNED | Primary key                  |
| name                    | VARCHAR(100)    | Unique display name          |
| code                    | VARCHAR(50)     | Unique machine-readable code |
| description             | VARCHAR(255)    | Role purpose                 |
| is_system_role          | BOOLEAN         | Protect built-in roles       |
| created_at / updated_at | DATETIME        | Timestamps                   |

- REQUESTER - creates and tracks own tickets.
- AGENT - handles assigned tickets and communicates with requesters.
- SUPERVISOR - manages queues, priorities, assignments and escalations.
- ADMIN - manages users, roles, departments, categories, knowledge base and settings.

### 5.3 permissions

| Column      | Type            | Example                |
|-------------|-----------------|------------------------|
| id          | BIGINT UNSIGNED | Primary key            |
| name        | VARCHAR(150)    | View all tickets       |
| code        | VARCHAR(100)    | ticket.view_all        |
| module      | VARCHAR(50)     | ticket                 |
| description | VARCHAR(255)    | Permission explanation |

### 5.4 Mapping tables

| Table            | Columns                                    | Constraint                     |
|------------------|--------------------------------------------|--------------------------------|
| user_roles       | user_id, role_id, assigned_by, assigned_at | UNIQUE(user_id, role_id)       |
| role_permissions | role_id, permission_id                     | UNIQUE(role_id, permission_id) |

Hiding an AdminLTE menu item is not security. Django must verify permissions on every protected view, API endpoint and service operation.

## 6. Organization and Classification

### 6.1 departments

| Column                  | Type            | Purpose                   |
|-------------------------|-----------------|---------------------------|
| id                      | BIGINT UNSIGNED | Primary key               |
| name                    | VARCHAR(150)    | Department name           |
| code                    | VARCHAR(50)     | Unique code, e.g. FINANCE |
| description             | TEXT NULL       | Description               |
| is_active               | BOOLEAN         | Controls availability     |
| created_at / updated_at | DATETIME        | Timestamps                |

### 6.2 ticket\_categories

| Column                  | Type                 | Purpose                    |
|-------------------------|----------------------|----------------------------|
| id                      | BIGINT UNSIGNED      | Primary key                |
| name                    | VARCHAR(150)         | Category name              |
| code                    | VARCHAR(50)          | Unique category code       |
| department_id           | BIGINT UNSIGNED NULL | Optional owning department |
| description             | TEXT NULL            | Category explanation       |
| is_active               | BOOLEAN              | Available for new tickets  |
| created_at / updated_at | DATETIME             | Timestamps                 |

Example categories: ACCOUNT\_ACCESS, FINANCE, TECHNICAL, ADMISSIONS, EXAMINATION and GENERAL\_INQUIRY.  
Store the AI-predicted category separately from the final category selected by the system or an authorized user.

## 7. Core Ticket Schema

### 7.1 tickets

| Column                  | Type                 | Purpose / Rule                                        |
|-------------------------|----------------------|-------------------------------------------------------|
| id                      | BIGINT UNSIGNED      | Primary key                                           |
| ticket_number           | VARCHAR(30)          | Unique public reference, e.g. RA-2026-000001          |
| requester_id            | BIGINT UNSIGNED      | FK users; ticket owner                                |
| department_id           | BIGINT UNSIGNED NULL | Owning department                                     |
| category_id             | BIGINT UNSIGNED NULL | Final selected category                               |
| subject                 | VARCHAR(255)         | Ticket title                                          |
| description             | LONGTEXT             | Initial problem description                           |
| priority                | VARCHAR(20)          | LOW, MEDIUM, HIGH, CRITICAL                           |
| status                  | VARCHAR(30)          | OPEN, IN_PROGRESS, WAITING_FOR_USER, RESOLVED, CLOSED |
| assigned_to_id          | BIGINT UNSIGNED NULL | Current primary agent                                 |
| resolved_at             | DATETIME NULL        | Resolution timestamp                                  |
| closed_at               | DATETIME NULL        | Closure timestamp                                     |
| created_at / updated_at | DATETIME             | Timestamps                                            |

For the internship MVP, Django TextChoices plus VARCHAR columns are preferable to MySQL ENUM because they remain easier to evolve through migrations and application code.

### 7.2 ticket\_messages

| Column                  | Type            | Purpose                                        |
|-------------------------|-----------------|------------------------------------------------|
| id                      | BIGINT UNSIGNED | Primary key                                    |
| ticket_id               | BIGINT UNSIGNED | FK tickets                                     |
| sender_id               | BIGINT UNSIGNED | FK users                                       |
| message_type            | VARCHAR(30)     | USER_REPLY, AGENT_REPLY, INTERNAL_NOTE, SYSTEM |
| body                    | LONGTEXT        | Message content                                |
| is_internal             | BOOLEAN         | Internal notes are hidden from requester       |
| created_at / updated_at | DATETIME        | Timestamps                                     |

### 7.3 ticket\_status\_history

| Column        | Type                 | Purpose                    |
|---------------|----------------------|----------------------------|
| id            | BIGINT UNSIGNED      | Primary key                |
| ticket_id     | BIGINT UNSIGNED      | FK tickets                 |
| from_status   | VARCHAR(30) NULL     | Previous state             |
| to_status     | VARCHAR(30)          | New state                  |
| changed_by_id | BIGINT UNSIGNED NULL | User or NULL for system    |
| reason        | TEXT NULL            | Optional transition reason |
| created_at    | DATETIME             | Transition timestamp       |

### 7.4 ticket\_assignments

| Column         | Type            | Purpose                   |
|----------------|-----------------|---------------------------|
| id             | BIGINT UNSIGNED | Primary key               |
| ticket_id      | BIGINT UNSIGNED | FK tickets                |
| assigned_to_id | BIGINT UNSIGNED | Agent                     |
| assigned_by_id | BIGINT UNSIGNED | Supervisor/admin          |
| assigned_at    | DATETIME        | Assignment timestamp      |
| unassigned_at  | DATETIME NULL   | End of assignment         |
| is_current     | BOOLEAN         | Current assignment marker |

Never delete status or assignment history to clean up a ticket. The tickets table stores the current state for fast access; history tables preserve the permanent operational trail.

## 8. AI/ML Data Model

AI outputs must be stored with model/version details, confidence, raw structured output and execution status. This supports debugging, auditing and later accuracy evaluation.

### 8.1 ai\_analyses

| Column           | Type              | Purpose                                      |
|------------------|-------------------|----------------------------------------------|
| id               | BIGINT UNSIGNED   | Primary key                                  |
| ticket_id        | BIGINT UNSIGNED   | FK tickets                                   |
| analysis_type    | VARCHAR(50)       | CLASSIFICATION, PRIORITY, SENTIMENT, SUMMARY |
| model_name       | VARCHAR(150)      | Model identifier                             |
| model_version    | VARCHAR(100) NULL | Version identifier                           |
| input_hash       | VARCHAR(64) NULL  | Optional deduplication/audit hash            |
| result_json      | JSON              | Structured AI output                         |
| confidence_score | DECIMAL(5,4) NULL | 0.0000 to 1.0000                             |
| status           | VARCHAR(30)       | SUCCESS, FAILED, LOW_CONFIDENCE              |
| error_message    | TEXT NULL         | Failure details                              |
| created_at       | DATETIME          | Analysis timestamp                           |

### 8.2 ai\_suggestions

| Column                 | Type                 | Purpose                 |
|------------------------|----------------------|-------------------------|
| id                     | BIGINT UNSIGNED      | Primary key             |
| ticket_id              | BIGINT UNSIGNED      | FK tickets              |
| created_by_analysis_id | BIGINT UNSIGNED NULL | FK ai_analyses          |
| suggestion_type        | VARCHAR(50)          | REPLY, FAQ, ACTION      |
| suggested_text         | LONGTEXT             | AI-generated suggestion |
| source_json            | JSON NULL            | Knowledge-base sources  |
| confidence_score       | DECIMAL(5,4) NULL    | Confidence              |
| used_by_id             | BIGINT UNSIGNED NULL | Agent who used it       |
| used_at                | DATETIME NULL        | Usage timestamp         |
| created_at             | DATETIME             | Creation timestamp      |

AI prediction is not the final business value. Store AI category and priority recommendations in ai\_analyses, while tickets.category\_id and tickets.priority represent the final operational values.

## 9. Knowledge Base and RAG Data

### 9.1 knowledge\_articles

| Column                  | Type                 | Purpose                    |
|-------------------------|----------------------|----------------------------|
| id                      | BIGINT UNSIGNED      | Primary key                |
| title                   | VARCHAR(255)         | Article title              |
| slug                    | VARCHAR(255)         | Unique URL identifier      |
| category_id             | BIGINT UNSIGNED NULL | Related category           |
| status                  | VARCHAR(30)          | DRAFT, PUBLISHED, ARCHIVED |
| created_by_id           | BIGINT UNSIGNED      | Author                     |
| updated_by_id           | BIGINT UNSIGNED NULL | Last editor                |
| published_at            | DATETIME NULL        | Publication time           |
| created_at / updated_at | DATETIME             | Timestamps                 |

### 9.2 knowledge\_article\_versions

| Column         | Type            | Purpose               |
|----------------|-----------------|-----------------------|
| id             | BIGINT UNSIGNED | Primary key           |
| article_id     | BIGINT UNSIGNED | FK knowledge_articles |
| version_number | INT UNSIGNED    | Sequential version    |
| content        | LONGTEXT        | Article body          |
| created_by_id  | BIGINT UNSIGNED | Editor                |
| created_at     | DATETIME        | Version timestamp     |

For the 8-week MVP, MySQL should remain authoritative for article metadata and content. If a vector database is introduced later, store the source article/version ID in vector metadata so AI answers remain traceable.

## 10. Supporting Tables

### 10.1 ticket\_attachments

| Column         | Type            | Purpose                          |
|----------------|-----------------|----------------------------------|
| id             | BIGINT UNSIGNED | Primary key                      |
| ticket_id      | BIGINT UNSIGNED | FK tickets                       |
| uploaded_by_id | BIGINT UNSIGNED | FK users                         |
| original_name  | VARCHAR(255)    | Original filename                |
| storage_key    | VARCHAR(500)    | File/object path; not raw binary |
| mime_type      | VARCHAR(100)    | Detected MIME type               |
| file_size      | BIGINT UNSIGNED | Bytes                            |
| created_at     | DATETIME        | Upload timestamp                 |

### 10.2 audit\_logs

| Column      | Type                 | Purpose                 |
|-------------|----------------------|-------------------------|
| id          | BIGINT UNSIGNED      | Primary key             |
| actor_id    | BIGINT UNSIGNED NULL | User/system actor       |
| action      | VARCHAR(100)         | Example: ticket.created |
| entity_type | VARCHAR(100)         | Example: Ticket         |
| entity_id   | BIGINT UNSIGNED NULL | Affected record         |
| old_values  | JSON NULL            | Before state            |
| new_values  | JSON NULL            | After state             |
| ip_address  | VARCHAR(45) NULL     | IPv4/IPv6               |
| user_agent  | TEXT NULL            | Client details          |
| created_at  | DATETIME             | Event timestamp         |

### 10.3 notifications

| Column      | Type              | Purpose              |
|-------------|-------------------|----------------------|
| id          | BIGINT UNSIGNED   | Primary key          |
| user_id     | BIGINT UNSIGNED   | Recipient            |
| type        | VARCHAR(50)       | Notification type    |
| title       | VARCHAR(255)      | Short title          |
| message     | TEXT              | Notification content |
| entity_type | VARCHAR(100) NULL | Related entity       |

| Column     | Type                 | Purpose            |
|------------|----------------------|--------------------|
| entity_id  | BIGINT UNSIGNED NULL | Related record     |
| read_at    | DATETIME NULL        | Read timestamp     |
| created_at | DATETIME             | Creation timestamp |

## 11. End-to-End Ticket Database Workflow

| Step | Action                                | Database Effect                                  |
|------|---------------------------------------|--------------------------------------------------|
| 1    | Requester submits ticket              | INSERT tickets with status OPEN                  |
| 2    | Initial history created               | INSERT ticket_status_history: NULL -> OPEN       |
| 3    | AI classification runs                | INSERT ai_analyses: CLASSIFICATION               |
| 4    | AI priority runs                      | INSERT ai_analyses: PRIORITY                     |
| 5    | AI sentiment and summary run          | INSERT ai_analyses records                       |
| 6    | Django stores final category/priority | UPDATE tickets.category_id and priority          |
| 7    | Supervisor assigns agent              | INSERT ticket_assignments; UPDATE assigned_to_id |
| 8    | Agent begins work                     | UPDATE status IN PROGRESS; INSERT history        |
| 9    | Agent replies                         | INSERT ticket_messages                           |
| 10   | AI suggests answer                    | INSERT ai_suggestions; agent reviews             |
| 11   | Requester responds                    | INSERT ticket_messages                           |
| 12   | Agent resolves issue                  | UPDATE RESOLVED and resolved_at; INSERT history  |
| 13   | Requester confirms or timeout occurs  | UPDATE CLOSED and closed_at; INSERT history      |
| 14   | Important actions audited             | INSERT audit_logs                                |

## 12. Ticket State Machine

```

OPEN
|
v
IN_PROGRESS <-----+
|                   |
v                   |
WAITING_FOR_USER --+
|
v
RESOLVED
|
v
CLOSED

```

Allowed transitions:

```

OPEN -> IN_PROGRESS
IN_PROGRESS -> WAITING_FOR_USER
WAITING_FOR_USER -> IN_PROGRESS
IN_PROGRESS -> RESOLVED
RESOLVED -> CLOSED
RESOLVED -> IN_PROGRESS    (reopen)
CLOSED -> IN_PROGRESS      (admin-only exceptional reopen)

```

Every transition must create a ticket\_status\_history row. The current status on tickets is the fast-access current state; history is the permanent audit trail.

## 13. Constraints, Indexes and Data Integrity

### 13.1 Required unique constraints

- users.email
- users.username
- tickets.ticket\_number
- departments.code
- ticket\_categories.code
- roles.code
- permissions.code
- knowledge\_articles.slug
- UNIQUE(user\_id, role\_id) on user\_roles
- UNIQUE(role\_id, permission\_id) on role\_permissions
- UNIQUE(article\_id, version\_number) on knowledge\_article\_versions

### 13.2 Recommended indexes

| Table                 | Index                                  | Reason                   |
|-----------------------|----------------------------------------|--------------------------|
| tickets               | (requester_id, created_at)             | Requester ticket history |
| tickets               | (status, priority)                     | Queue filtering          |
| tickets               | (department_id, status)                | Department queue         |
| tickets               | (assigned_to_id, status)               | Agent workload           |
| tickets               | (category_id, created_at)              | Category analytics       |
| ticket_messages       | (ticket_id, created_at)                | Conversation order       |
| ticket_status_history | (ticket_id, created_at)                | Timeline                 |
| ticket_assignments    | (ticket_id, is_current)                | Current assignment       |
| ai_analyses           | (ticket_id, analysis_type, created_at) | Latest AI result         |
| audit_logs            | (entity_type, entity_id, created_at)   | Entity audit trail       |
| notifications         | (user_id, read_at, created_at)         | Unread notifications     |

### 13.3 Deletion policy

- Deactivate users rather than deleting them.
- Protect tickets from requester deletion.
- Retain messages, status history, assignment history and audit records.
- Deactivate categories and departments instead of deleting referenced records.
- Archive knowledge articles instead of deleting them.
- Delete attachment files only through an explicit retention policy.

## 14. Recommended Django Model Organization

```
apps/
|-- accounts/      # User, Role, Permission, mappings
|-- organization/  # Department
|-- tickets/       # Ticket, Message, StatusHistory, Assignment
|-- classification/ # TicketCategory
|-- ai/            # AIAnalysis, AISuggestion, FastAPI client
|-- knowledge/     # Article, ArticleVersion
|-- audit/         # AuditLog
|-- notifications/ # Notification
```

Avoid one giant models.py. Organize models by domain, keep business workflow rules in service functions/classes, and keep read/query logic separate when practical.

## 15. Example Django Ticket Model Shape

```
class Ticket(models.Model):
    ticket_number = models.CharField(max_length=30, unique=True)
    requester = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name='submitted_tickets',
```

```
)
department = models.ForeignKey(
    Department, null=True, blank=True,
    on_delete=models.SET_NULL,
)
category = models.ForeignKey(
    TicketCategory, null=True, blank=True,
    on_delete=models.SET_NULL,
)
assigned_to = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    null=True, blank=True,
    on_delete=models.SET_NULL,
    related_name='assigned_tickets',
)
subject = models.CharField(max_length=255)
description = models.TextField()
priority = models.CharField(max_length=20, choices=Priority.choices)
status = models.CharField(max_length=30, choices=Status.choices)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
```

## 16. Django-FastAPI Database Boundary

| Operation        | Owner   | Database Interaction                      |
|------------------|---------|-------------------------------------------|
| Create ticket    | Django  | INSERT tickets                            |
| Update status    | Django  | UPDATE tickets + INSERT history           |
| Assign agent     | Django  | INSERT assignment + UPDATE ticket         |
| Add message      | Django  | INSERT ticket_messages                    |
| Run AI analysis  | FastAPI | No direct core-table write; return result |
| Store AI result  | Django  | INSERT ai_analyses                        |
| Suggest answer   | FastAPI | Search/compute and return result          |
| Store suggestion | Django  | INSERT ai_suggestions                     |
| Audit action     | Django  | INSERT audit_logs                         |

Recommended AI response contract:

```
{
  "success": true,
  "analysis_type": "CLASSIFICATION",
  "model_name": "ticket-classifier",
  "model_version": "v1",
  "result": {"category_code": "ACCOUNT_ACCESS"},
  "confidence_score": 0.91
}
```

## 17. Initial Seed Data

| Type               | Initial Records                                                              |
|--------------------|------------------------------------------------------------------------------|
| Roles              | REQUESTER, AGENT, SUPERVISOR, ADMIN                                          |
| Departments        | IT_SUPPORT, FINANCE, ADMISSIONS, EXAMINATION, GENERAL                        |
| Categories         | ACCOUNT_ACCESS, FINANCE, TECHNICAL, ADMISSIONS, EXAMINATION, GENERAL_INQUIRY |
| Demo users         | One requester, two agents, one supervisor, one admin                         |
| Knowledge articles | At least 10 realistic FAQ articles                                           |
| Demo tickets       | At least 20 tickets across statuses, categories and priorities               |

Use a Django management command or fixtures for deterministic seed data. Developers should not need to run manual SQL inserts on every machine.

## 18. Transactional Workflows

Actions that update several tables must be atomic. Example: assigning a ticket.

```
1. Validate ticket.assign permission
2. Close previous current assignment, if any
3. Insert ticket_assignments row
4. Update tickets.assigned_to_id
5. Optionally change OPEN -> IN_PROGRESS
6. Insert ticket_status_history if status changed
7. Insert audit_log
8. Commit all changes
```

Use Django transaction.atomic() for multi-table operations. If any required step fails, roll back the complete operation.

## 19. Security and Privacy Rules

- Use Django password hashing; never store plain-text passwords.
- Keep database credentials and secrets in environment variables.
- Requesters can view only their own tickets.
- Agents can access assigned tickets and explicitly permitted queues.
- Supervisors manage authorized department queues.
- Internal notes must never be exposed to requesters.
- Validate attachment type, extension and size.
- Do not log passwords, tokens or unnecessary sensitive message content.
- Audit permission changes, assignments, status changes and administrative actions.

## 20. Performance Guidelines

- Use select\_related() for foreign-key-heavy ticket lists.
- Use prefetch\_related() for messages and history when needed.
- Paginate ticket lists and conversations.
- Add indexes based on real query patterns.
- Avoid loading complete conversation histories on dashboard pages.
- Store large files outside MySQL; keep metadata and storage keys only.
- For the MVP, synchronous AI calls are acceptable for small workloads, but design services so background jobs can be introduced later.

## 21. Intern Implementation Plan

| Phase | Work                 | Expected Output                          |
|-------|----------------------|------------------------------------------|
| 1     | ERD and requirements | Approved conceptual and logical ERD      |
| 2     | Custom user and RBAC | Users, roles, permissions and migrations |
| 3     | Organization         | Departments and categories               |
| 4     | Ticket core          | Tickets, messages, status history and    |

| Phase | Work                    | Expected Output                           |
|-------|-------------------------|-------------------------------------------|
|       |                         | assignments                               |
| 5     | AI persistence          | AI analyses and suggestions               |
| 6     | Knowledge base          | Articles and versions                     |
| 7     | Supporting modules      | Attachments, audit logs and notifications |
| 8     | Indexes and constraints | Optimized and validated schema            |
| 9     | Seed data               | Repeatable demo dataset                   |
| 10    | Testing                 | Model, RBAC and workflow tests            |

## 22. Database Definition of Done

- All required tables exist through Django migrations.
- A fresh developer can initialize MySQL from zero using documented commands.
- Foreign keys and unique constraints are implemented.
- RBAC is represented in the database and enforced by Django.
- Ticket status transitions create history records.
- Assignments preserve historical records.
- AI predictions are stored independently from final business values.
- Internal notes are protected by authorization.
- Seed data is repeatable and creates at least 20 realistic demo tickets.
- Critical workflow operations use transactions.
- Required indexes are present and justified.
- Every intern can explain the ERD, tables and important relationships.

## 23. Required Team Deliverables

- ERD in Draw.io, Lucidchart, Mermaid or an equivalent tool.
- Django models organized by domain applications.
- All migrations committed to GitLab.
- One documented command to load seed data.
- Data dictionary explaining every table and important column.
- At least 10 operational Django ORM or SQL queries.
- Tests covering CRUD, RBAC, ticket lifecycle and AI persistence.
- README containing complete MySQL setup and migration instructions.

## 24. Final Implementation Sequence

Finalize ERD -> Mentor review -> Create Django apps/models -> Generate migrations -> Run migrations against MySQL -> Load seed data -> Verify relationships through Django Admin -> Implement ticket lifecycle -> Integrate FastAPI results -> Add tests.

The objective is not to build the most complicated database. The objective is to build a clean, explainable and extensible schema that correctly represents the end-to-end support workflow and preserves the distinction between AI recommendations and human-controlled business decisions.